Súčasné trendy vo vývoji automatizovaných systémov s využitím asynchrónneho programovania v jazyku Python*

Oleksii Kovalevskyi

Slovenská technická univerzita v Bratislave

Fakulta informatiky a informačných technológií

xkovalevskyi@stuba.sk

00000000

28. september 2026

School of Hard Knocks
SOCI4568 L01 Sociology of Physics
For: Professor Y.R.U StillHere

Abstrakt

Tento článok poskytuje prehľad súčasného stavu v oblasti vývoja automatizovaných softvérových botov s využitím asynchrónneho programovania. Zameriava sa na moderné prístupy a knižnice v jazyku Python, ako sú aiogram a apscheduler, ich architektúru, výhody a úskalia pri nasadzovaní v reálnych aplikáciách. Cielom je zosumarizovať dostupné riešenia a porovnať ich efektivitu.

1 Úvod

Asynchrónne programovanie v jazyku Python predstavuje moderný prístup k vývoju softvéru, ktorý umožňuje efektívne spracovanie veľkého množstva paralelných úloh. Tento článok sa zameriava na knižnice ako aiogram a apscheduler. Použitie takýchto nástrojov je kľúčové pre tvorbu rýchlych a škálovateľných automatizovaných botov. Tento odsek bol doplnený tak, aby bolo viditeľné plynulé obtekanie textu okolo vloženého plávajúceho loga fakulty, presne podľa požiadaviek cvičenia.

Obr. 1: Logo FIIT STU

2 Prehľad architektúry asynchrónnych systémov

V tejto časti analyzujeme základnú štruktúru bota. Na nasledujúcich diagramoch môžeme vidieť rozdelenie komponentov a tok spracovania správ. Nasledujúce dva diagramy sú umiestnené vedľa seba.

3 Analýza moderných nástrojov v jazyku Python

Knižnice aiogram a apscheduler poskytujú robustné API pre vývojárov. Vďaka event-loopu v Pythone nedochádza k blokovaniu hlavného vlákna pri čakaní na odpoveď zo servera Telegramu. Pre detailný pohľad uvádzame široký diagram prekrývajúci oba stĺpce.

4 Porovnanie a zhodnotenie prístupov

Pri porovnávaní výkonnosti asynchrónnych systémov môžeme stavy jednotlivých taskov (úloh) vizualizovať pomocou stavovej matice s rozmerom 5x4:

*Semestrálny projekt v predmete Metódy inžinierskej práce, ak. rok 2026/27, vedenie: Richard Marko

$$\begin{bmatrix} task_{11} & task_{12} & task_{13} & task_{14} \\ task_{21} & task_{22} & task_{23} & task_{24} \\ task_{31} & task_{32} & task_{33} & task_{34} \\ task_{41} & task_{42} & task_{43} & task_{44} \\ task_{51} & task_{52} & task_{53} & task_{54} \end{bmatrix}$$

Na výpočet celkovej časovej zložitosti a odozvy systému pri veľkom zaťažení (napríklad pri plánovaní stoviek úloh) používame nasledujúci rozvoj, ktorý svojou dĺžkou presahuje šírku jedného stĺpca dokumentu, a preto je zalomený do viacerých riadkov:

$$T_{total} = t_{init} + \sum_{i=1}^{n}(p_i \cdot w_i) + c_{db} + c_{api} + c_{cache} + \\ + \delta_{net1} + \delta_{net2} + \delta_{net3} + \delta_{net4} + \delta_{net5} + \mathcal{O}(N \log N) \tag{1}$$

5 Záver

Asynchrónne systémy v Pythone predstavujú efektívne riešenie pre moderné aplikácie. V ďalšej práci na prehľadovom článku sa zameriame na hlbšiu optimalizáciu a možnosti nasadenia v produkčnom prostredí s ohľadom na dostupné zdroje.

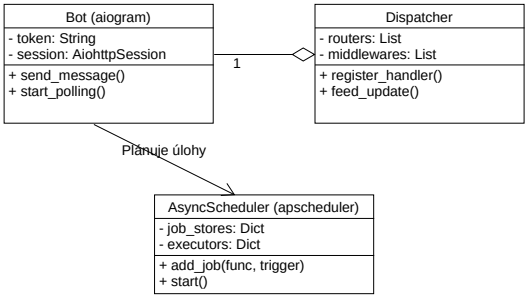
Literatúra

[CHE05] Krzysztof Czarnecki, Simon Helsen, and Ulrich Eisenecker. Staged configuration through specialization and multi-level configuration of feature models. *Software Process: Improvement and Practice*, 10:143–169, April/June 2005.

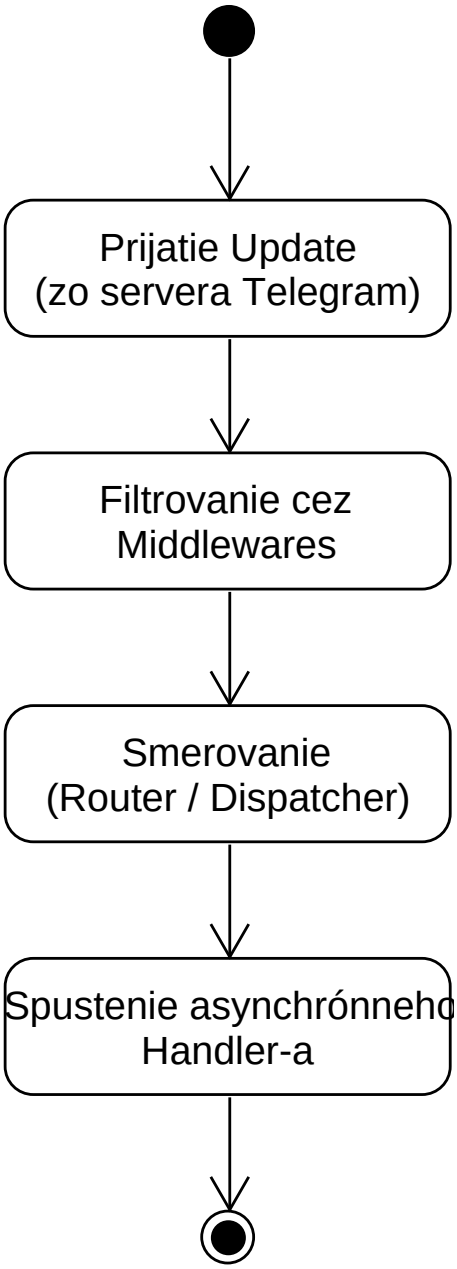
[CK05] Krzysztof Czarnecki and Chang Hwan Peter Kim. Cardinality-based feature modeling and constraints: A progress report. In *International Workshop on Software Factories, OOPSLA 2005*, San Diego, USA, October 2005.

[Cop99] James O. Coplien. *Multi-Paradigm Design for C++*. Addison-Wesley, 1999.

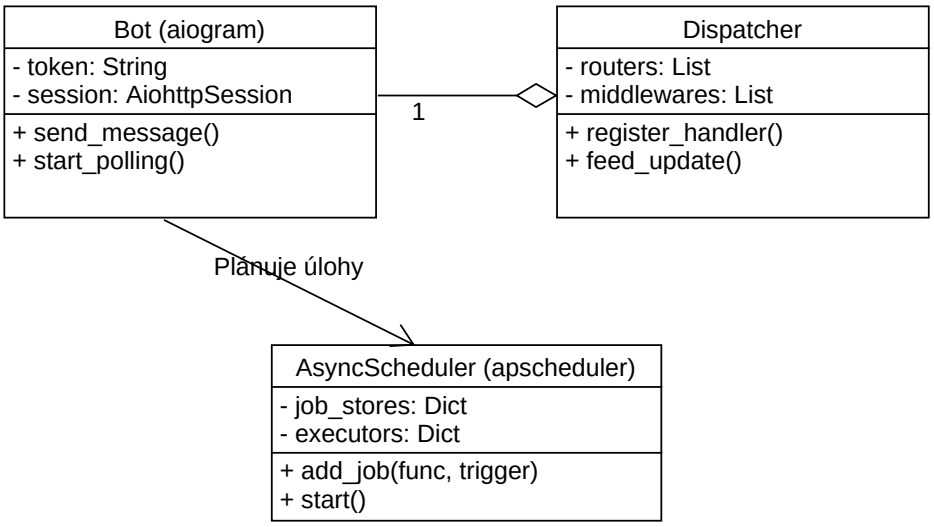
[SEI] Carnegie Mellon University Software Engineering Institute. A framework for software product line practice—version 5.0. `http://www.sei.cmu.edu/productlines/frame_report/`.



Obr. 2: Architektúra Python bota



Obr. 3: Spracovanie updatov



Obr. 4: Celkový pohľad na asynchrónny systém prekrývajúci obidva stĺpce